Technical Assignment — Internship Application

Flight Management Web App (PWA)

Deadline: Sunday, 5:00 PM
Submission: GitHub repository link
Type: Individual

Overview

Build a Flight Management web application — a responsive, production-like app where passengers can search and book flights, select seats, reschedule, and cancel bookings. The primary deliverable is a **fully responsive web app**. As a bonus, configure it as an installable **Progressive Web App (PWA)**.

Tech Stack

- **Frontend & API:** Next.js 14+ (App Router)
- **Database & Auth:** Supabase (PostgreSQL + Supabase Auth + Realtime)
- **State Management:** Zustand with `persist` middleware
- **Styling:** Tailwind CSS
- **PWA (bonus):** next-pwa

Database Schema (Supabase)

Design and migrate the following tables in your Supabase project. Include all SQL files in a `/supabase/migrations` folder.

Tables required:

- `flights` — id, flight_no, origin, destination, departs_at, arrives_at, aircraft_type, status, base_price
- `seats` — id, flight_id, seat_number, class (economy/business/first), is_available, extra_fee
- `bookings` — id, user_id, flight_id, seat_id, status, booked_at, total_price, pnr_code

- `passengers` — id, booking_id, full_name, passport_no, nationality, dob
- `reschedules` — id, booking_id, old_flight_id, new_flight_id, requested_at, fee_charged

Requirements:

- Enable **Row Level Security (RLS)** on all tables — users can only access their own bookings
- Create a **Supabase RPC function** for seat reservation to prevent double-booking race conditions
- Enforce a **DB-level constraint or trigger**: cancellations within 2 hours of departure must be rejected
- Seed the DB with at least 8 flights across 4 routes, with a full seat map per flight
- Use **Supabase Auth** for user management

Tasks

Task 01 — Flight Search & Booking Flow

Build the core booking journey from search to confirmation.

- Search page with origin, destination, date, and passenger count
- Results page listing matching flights with price, duration, and class options
- Booking form to collect passenger details (name, passport number, nationality)
- Confirmation page showing the generated **PNR code**, flight details, and seat assignment
- Use Supabase server-side client in Server Components — do not expose keys beyond the anon key
- Validate seat availability before inserting a booking using the RPC seat-lock function

---

Task 02 — Interactive Seat Selection

Build a visual aircraft seat map with live availability.

- Render a cabin grid (rows × columns) for the selected flight
- Color-code seats: available / selected / occupied / your seat
- Support economy, business, and first class zones with clear visual separation
- Subscribe to **Supabase Realtime** on the `seats` table — seats booked by other users update live without a page refresh
- Disabled occupied seats with a tooltip showing class and extra fee on hover
- Seat map must be scrollable and touch-friendly on mobile

Task 03 — Rescheduling & Cancellation

Allow users to manage their existing bookings.

- **My Bookings** page listing all bookings with status badges (confirmed / rescheduled / cancelled)
- **Reschedule:** user picks an alternative flight on the same route — insert into `reschedules`, update booking's `flight_id`, charge a fee if the new flight is more expensive
- **Cancel:** update booking status to `cancelled` and free the seat atomically using a single Supabase RPC
- Cancellations within 2 hours of departure must be blocked (enforced at DB level)
- Show a confirmation dialog before any destructive action

Task 04 — Zustand Store with `persist`

Design a well-structured Zustand store with proper persistence.

- Create `useFlightStore` with: active search query, selected flight, selected seat, current booking step, and passenger form data
- Use `persist` middleware to save the search query and in-progress booking so users can resume after closing the tab
- Use `partialize` in the persist config to **exclude sensitive fields** (passport numbers) from localStorage
- Create a separate `useUserStore` for the Supabase auth session and cached bookings — persist only the session token
- Implement **optimistic seat selection**: mark the seat as selected in the store before the Supabase write confirms
- Include a store reset action triggered on booking cancellation or logout

---

Task 05 — PWA (Bonus)

Make the app installable and offline-capable.

- Configure `next-pwa` with a valid `manifest.json`: app name, icons (192×192 and 512×512), theme colour, `display: standalone`
- Cache strategy: StaleWhileRevalidate for flight search results, CacheFirst for static assets
- Offline fallback page shown when there is no connectivity
- **My Bookings** page must be readable offline using last-cached data
- App should score ≥ 90 on Lighthouse PWA audit — include a screenshot in your README
- Add an install prompt banner for first-time mobile visitors

---

Evaluation Criteria

- **Schema & RLS** — Are policies correct? Is the seat-locking RPC safe against concurrency?
- **Seat map UX** — Is the grid intuitive? Does Realtime sync work correctly?
- **Reschedule & cancel logic** — Are atomic operations handled correctly? Is the 2-hour rule enforced at DB level?
- **Zustand store design** — Is `partialize` used correctly? Is sensitive data excluded? Is reset handled cleanly?
- **Responsive design** — Does the app work well on mobile, tablet, and desktop?
- **Code quality** — TypeScript throughout, no `any`, clean component separation, meaningful commit history

---

Submission Checklist

- [ ] Public GitHub repository with descriptive commit history
- [ ] `.env.example` with all Supabase environment variables listed
- [ ] Supabase migration SQL files in `/supabase/migrations`
- [ ] Seed script with flights, seats, and a test user account (share credentials in README)
- [ ] README with local setup steps, Supabase project config, and an explanation of your Zustand store structure
- [ ] Deployed preview link (Vercel recommended) — not required but appreciated
- [ ] Lighthouse PWA screenshot in README (bonus task)
- [ ] Deploy the application on Vercel and submit the production url as well

---

Deadline: Saturday, 5:00 PM
Submit your GitHub repository link before the deadline. If you run short on time for a feature, document what you would have done differently in your README. We care about how you think, how you structure complexity, and how honestly you communicate trade-offs. Good luck.